Ascend CANN 自定义算子开发 — 从零到入门学习路线图

> 本文档面向 **零基础** 的开发者，系统梳理学习 Ascend 自定义算子开发所需的知识、资源和实践路径。 配合本项目 (`aclnn_launch_example`) 使用效果最佳。

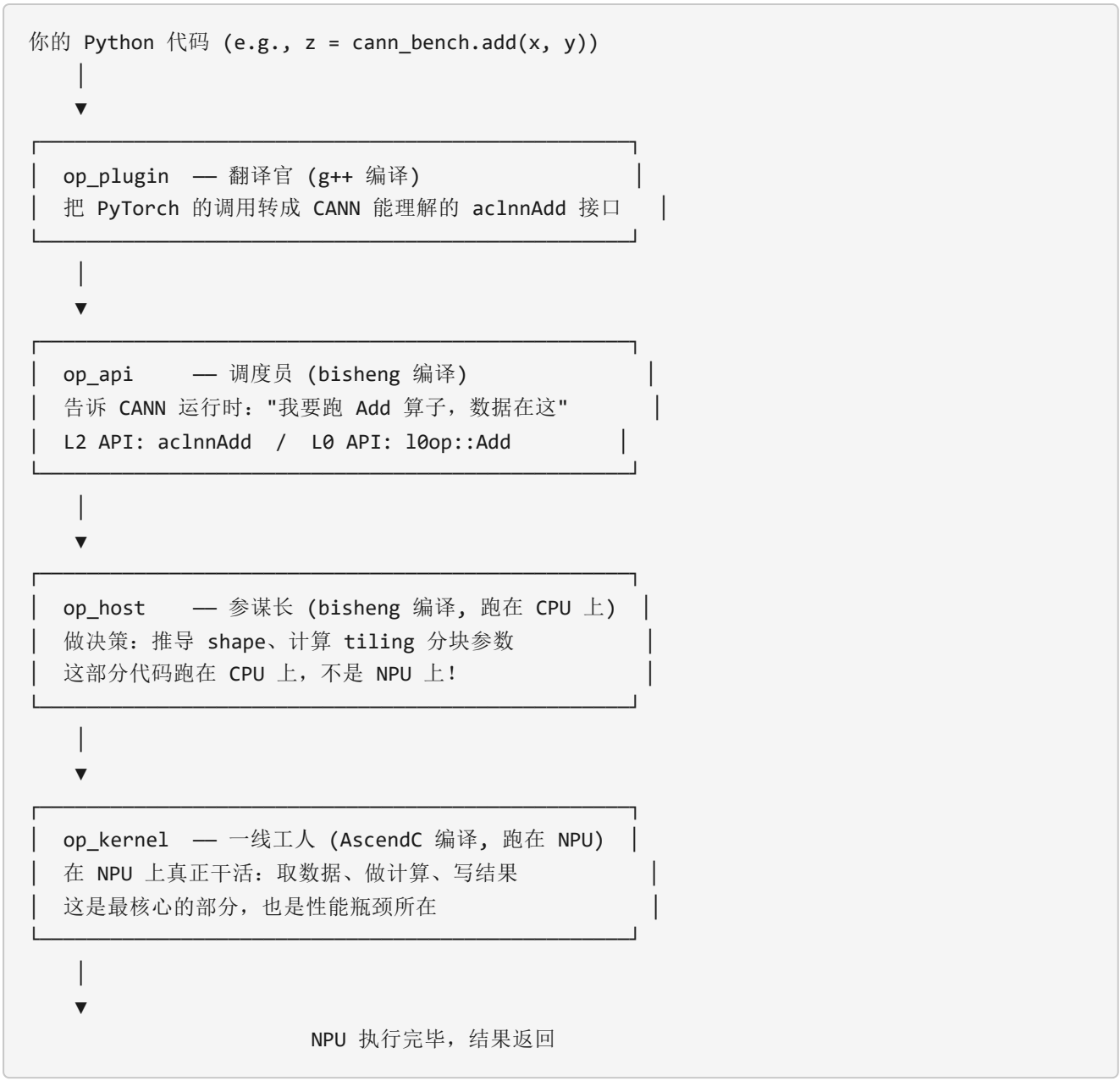
目录

- 一、为什么算子结构是 4 层？—— 先搞懂全貌
- 二、新手常见困惑
- 三、分阶段学习路线
  - 阶段 0：了解背景知识
  - 阶段 1：第一个 Ascend C 算子——动手跑通
  - 阶段 2：理解完整算子工程
  - 阶段 3：系统化学习与认证
  - 阶段 4：进阶与性能优化
- 四、针对本项目的学习顺序
- 五、参考资源汇总
- 六、常见问题

一、为什么算子结构是 4 层？—— 先搞懂全貌

算子执行全流程

一个自定义算子在 NPU 上执行，**不是一步到位的**，而是经过多层分工：



为什么一定要分 4 层？

层	跑在什么硬件上	用什么编译器	负责什么
op_plugin	CPU (Host)	g++	对接 PyTorch 框架
op_api	CPU (Host)	bisheng	封装算子调用接口
op_host	CPU (Host)	bisheng	形状推导、数据分块策略
op_kernel	NPU (Device)	AscendC	实际计算

核心原因：CPU 和 NPU 是两种不同的处理器，用不同的编译器，跑不同的代码。CPU 负责"动脑子"（决策调度），NPU 负责"干活"（并行计算）。所以代码必须分开，各司其职。

二、新手常见困惑

Q1：为什么同一个算子要写这么多文件？

因为 CANN 的构建系统 (`npu_op_code_gen`, `npu_op_library` 等 CMake 宏) 期望你按固定结构组织代码，才能自动生成中间文件、正确链接库。这不是"过度设计"，而是框架规范。

Q2：op_host 和 op_kernel 里都有 tiling，它们什么关系？

- **op_host 里的 tiling**：在 CPU 上决定"数据怎么分块"，比如一个 1024 长度的向量，分成 4 块，每块 256 个元素
- **op_kernel 里的 tiling**：在 NPU 上接收这些分块参数，按参数执行

即：op_host 计算分块方案，op_kernel 使用分块方案。

Q3：L2 API 和 L0 API 有什么区别？

- **L2 API** (`aclnnAdd`)：对外暴露的完整接口，包含 `GetWorkspaceSize`（计算所需临时空间）和实际执行函数
- **L0 API** (`l0op::Add`)：底层算子调用，由 L2 API 内部调用，普通用户不需要直接接触

三、分阶段学习路线

阶段 0：了解背景知识

目标：搞清楚 NPU 是什么、CANN 是什么、算子在 AI 框架中扮演什么角色。

预估时间：1-2 天

学习内容	资源	说明
昇腾 AI 处理器架构简介	昇腾社区	了解 AI Core、Cube Unit、Vector Unit 等基本概念
CANN 软件栈概述	神经网络算子开发入门	免费课程，了解 CANN 的分层架构
什么是算子	PyTorch 官方文档	理解 `torch.add`、`torch.mm` 这些"算子"在神经网络中的作用

阶段 0 的检验标准：能用自己的话解释"CPU 和 NPU 有什么区别"、“算子在 AI 计算中起什么作用”。

阶段 1：第一个 Ascend C 算子——动手跑通

目标：先动手，写一个最简单的 Add 算子并跑通。不要纠结于理解所有细节，先建立"我能写算子"的信心。

预估时间：3-5 天

步骤	资源	说明
1️⃣ 环境准备	昇腾文档	安装 CANN 工具包，确认 `npu-smi` 能识别 NPU
2️⃣ 第一个核函数	⭐ **我的第一份 Ascend C 代码**	**官方推荐起点**，手把手写 Add 算子
3️⃣ 快速入门教程	快速入门 - Ascend C 算子开发	官方文档中的完整流程
4️⃣ HelloWorld 样例	HelloWorld 样例	最简单的核函数示例

阶段 1 的检验标准：能独立写一个 `Add` 算子的核函数（`kernel_operator.h`），并调用 `kernel_ascendc` 工具跑通。

阶段 2：理解完整算子工程

目标：阶段 1 只是写了"裸核函数"，现在要理解完整的算子工程——包括算子定义、shape 推导、tiling、API 封装、Python 绑定。

预估时间：1 周

步骤	学习内容	资源	说明
1️⃣	算子定义 (op_def)	Ascend C 算子实现详解	以 Add 为案例，看完整实现
2️⃣	工程化开发流程	基于自定义算子工程的算子开发	从创建工程到运行验证
3️⃣	对照本项目	本项目 `aclnn_launch_example`	打开 `csrc/ops/add/` 目录，逐文件对照学习
4️⃣	从理论到实践	一文教你如何调用 Ascend C 算子	三种调用方式验证算子

阶段 2 的检验标准：能看懂本项目 `add/` 目录下每个文件的作用，并能仿照写一个 `Mul` 算子。

阶段 3：系统化学习与认证

目标：系统掌握 Ascend C 编程语言、CANN 算子开发全流程，获得官方认证。

预估时间：2-4 周

学习内容	资源	说明
在线课程	2025 昇腾 CANN 训练营	直播课程，适合不同阶段开发者
完整学习闭环	从入门到认证：Ascend C 算子开发完整学习闭环	从需求分析到高性能代码架构
技术原理	Ascend C 技术深度解析	深入理解编程模型和抽象硬件架构
算子注册与部署	昇腾算子注册与模型迁移	6 章教程，覆盖迁移到部署
官方认证	Ascend C 算子开发能力认证（初级）	实操考试，检验学习成果

阶段 3 的检验标准：获得 Ascend C 算子开发初级认证，能独立完成一个简单自定义算子的完整开发。

阶段 4：进阶与性能优化

目标：深入理解高性能算子设计，掌握多核并行、DoubleBuffer、Tiling 切分等优化技术。

学习内容	资源
多核 & Tiling 切分	昇腾文档 - 进阶教程
DoubleBuffer 机制	昇腾文档 - 性能优化
Memory 层级与数据搬运	昇腾文档 - 编程模型
复杂算子实现	社区博客、昇腾论坛

四、针对本项目的学习顺序

你现在手上有 `aclnn_launch_example` 项目，建议按以下顺序阅读代码：

```
第 1 步：csrc/ops/add/op_kernel/add_kernel.cpp      ← 最核心：NPU 上怎么算的
第 2 步：csrc/ops/add/op_kernel/add_tiling_data.h   ← tiling 数据结构
第 3 步：csrc/ops/add/op_host/add_def.cpp            ← 算子定义：输入输出类型
第 4 步：csrc/ops/add/op_host/add_infershape.cpp     ← shape 推导
第 5 步：csrc/ops/add/op_host/add_tiling.cpp         ← tiling 分块策略
第 6 步：csrc/ops/add/op_api/aclnn_add.cpp           ← L2 对外接口
第 7 步：csrc/ops/add/op_api/add.cpp                 ← L0 底层接口
第 8 步：csrc/ops/add/op_plugin/add_plugin.cpp       ← Python 绑定
第 9 步：csrc/ops/add/CMakeLists.txt                 ← 算子自注册
第 10 步：CMakeLists.txt (顶层)                      ← 整体构建流程
```

从核心往外看——先理解 NPU 上真正干了什么，再一层层看懂封装，比从 Python 往 NPU 方向看更容易理解。

看懂 `add/` 后，再看 `sqrt/` 作为对照，两个算子都看懂了，你就掌握了这个项目。

五、参考资源汇总

官方文档

资源	链接
昇腾文档中心（所有版本入口）	https://www.hiascend.com/document
快速入门 - Ascend C 算子开发 (CANN 8.3.RC1)	查看
基于自定义算子工程的算子开发 (CANN 8.0.RC3)	查看
Ascend C 算子开发接口 (API 参考)	查看

官方教程 & 文章

资源	链接
⭐ 我的第一份 Ascend C 代码	查看
⭐ Ascend C 算子实现详解（Add 案例）	查看
Ascend C 技术深度解析	查看
神经网络算子开发入门（免费课程）	查看

训练营 & 认证

资源	链接
2025 昇腾 CANN 训练营	查看
Ascend C 算子开发能力认证（初级）	查看
从入门到认证：完整学习闭环	查看

社区 & 论坛

资源	链接
昇腾社区	https://www.hiascend.com
华为开发者论坛	https://developer.huawei.com/home/forum/
知乎专栏 - 昇腾 CANN 算子开发	查看

六、常见问题

学这个需要什么基础？

- **必备**：C/C++ 基础（指针、内存管理）、Linux 基础操作
- **推荐**：了解 PyTorch 基本使用、了解并行计算概念
- **不需要**：不需要了解 NPU 硬件细节，不需要汇编

需要什么硬件？

需要一个昇腾 NPU 设备（如 Atlas 系列推理卡、昇腾 910B 训练卡），或云上昇腾实例。

全部学完大概要多久？

按照上述路线，从零到能独立开发简单自定义算子，**大约 3-6 周**（每天投入 2-3 小时）。

看文档还是看视频？

建议**先看文档动手**，遇到不懂的概念再找视频补充理解。动手写代码是学习效率最高的方式。

> **最后给一句话建议：**先跑通“我的第一份 Ascend C 代码”，再回头看你这个工程，就会豁然开朗。